

Périmètre produit et arbitrages : MVP, optionnel, backlog

Un projet évalué se juge autant sur ce qu'il **livre** que sur ce qu'il **écarte en le sachant**. `README.md` fixe le périmètre ; `docs/BACKLOG.md` garde la trace de ce qui a été volontairement différé, « avec le contexte, pour ne pas le perdre ». Savoir justifier un choix de scope à l'oral, c'est être capable de dire : ce qui manque, pourquoi, et ce que ça coûterait de le faire.

MVP obligatoire et optionnel

MVP (Minimum Viable Product) : la plus petite version qui remplit le besoin de base.

Périmètre	User stories (US)	Contenu	État (README)
MVP obligatoire	US01 → US06	upload avec compte, téléchargement par lien, création de compte, connexion, historique, suppression	livré et testé (unitaire, intégration, e2e)
Optionnel	US07	upload anonyme	non traité
	US08	tags	non traité
	US09	mot de passe sur un fichier	livré
	US10	expiration automatique	partiel : purge écrite et testée, déclenchement manuel

Le README annonce lui-même cet état : c'est un tableau d'honnêteté, pas une liste de promesses.

Plateformes : navigateur de bureau et iPhone (Safari iOS) en application web responsive, pas d'application native. Un job Cypress WebKit simule l'iPhone ; un passage manuel sur un iPhone réel reste à faire (point signalé comme tel).

Anatomie d'un arbitrage : le cas US10

Le backlog structure chaque décision de la même façon. Exemple, l'expiration automatique :

1. **État actuel** : un fichier expiré reste en base et dans MinIO ; l'UI affiche « Ce fichier a expiré » ; le nettoyage se fait à la main (`deploy/purge-expired.sh`).
2. **Ce qu'il manque exactement** : la logique existe et est testée (`ExpiredFilePurgerIT`) ; US10 n'ajoute que le déclencheur, un `@Scheduled` .
3. **Pourquoi c'est hors MVP** : la purge manuelle suffit pour une évaluation.
4. **Ce qui le rendrait nécessaire** : les comptes abandonnés dont le propriétaire ne revient jamais purger sa liste.
5. **Où ce sera fait** : avec la PR de déploiement/exploitation.

Le coût réel de US10 est faible parce que la fonctionnalité a été **découpée** : la logique est livrée et testée, seul le branchement est reporté.

Ce qui a été écarté, et pourquoi

Sujet	Décision	Raison donnée
Rotation de la clé JWT (JSON Web Token)	non planifié	tourner le secret HMAC toutes les 24 h invalide tous les tokens en cours, sauf jeu de clés à validité recouvrante (<code>kid</code>) ; avec des tokens courts et sans refresh token, très perturbant
Refresh token	hors MVP	re-login à l'expiration (1 h) ; rotation, révocation et stockage httpOnly sont un ajout naturel après
Antivirus (ClamAV)	hors MVP	listé hors périmètre
<code>tika-parsers</code> (détection <code>.msi</code> , <code>.jar</code>)	non fait	dépendance nettement plus lourde pour un gain limité à 2 des 17 extensions de la liste noire ; à arbitrer sur signal d'abus réel
SpotBugs	écarté, essai fait	9 findings dont 8 de bruit (<code>EI_EXPOSE_REP*</code> , 89 %), un seul signal réel (<code>DM_EXIT</code>)
Pagination de <code>GET /api/files</code>	différée	suffisant tant que le nombre de fichiers par compte est faible (fichiers expirés en ≤ 7 jours)
Lien à usage unique, reprise de téléchargement	différés	le lien reste valide jusqu'à expiration, comme un lien WeTransfer
nginx gzip + cache (PR #108)	mise de côté	PR prête, mais Lighthouse est déjà à 99/100 sans elle
<code>docker-compose.prod.yml</code>	hors MVP	le MVP porte sur les fonctionnalités, pas sur le déploiement ; les deux Dockerfile existent déjà

Le cas de la rotation de clé JWT, à savoir défendre

L'idée venait du Projet 2 (contrôle d'âge de la clé à 24 h). Le backlog demande un « point de vue critique » avant toute implémentation :

- Rotation quotidienne + tokens de 15 à 60 min + pas de refresh token = déconnexions fréquentes.
- Alternatives plus simples : secret stable géré par variable d'environnement ou gestionnaire de secrets ; ou RS256 + endpoint JWKS (JSON Web Key Set) avec rotation propre.
- Conclusion du backlog : pour un MVP évalué, un secret stable bien géré suffit ; la rotation est un sujet d'exploitation qui peut se documenter sans code.

Ce que fait le projet : le secret vient de l'environnement en prod (`DATASHARE_JWT_SECRET`) ; changer sa valeur déconnecte tout le monde (`MAINTENANCE.md`). La rotation propre reste au backlog.

Comment justifier un choix de scope à l'oral

Un schéma en quatre temps, que le backlog illustre à chaque entrée :

1. **Nommer** ce qui n'est pas fait, sans détour.
2. **Donner la raison** : coût, risque, valeur limitée, dépendance à autre chose.
3. **Montrer le filet** : ce qui protège en attendant (purge manuelle, liste noire par nom, secret stable en environnement).
4. **Dire le déclencheur** : ce qui ferait passer le sujet en priorité (signal d'abus réel, image utilisée en prod, comptes abandonnés).

Dans le même esprit, plusieurs arbitrages sont **datés et conditionnels** : Trivy non bloquant tant que `silo` ne sert pas en prod ; job WebKit non requis tant qu'il n'est pas jugé stable.

Dans DataShare

- Périmètre et état : `README.md` (sections « Périmètre » et en-tête).
- Carte détaillée US → code, avec statuts : `DESIGN.md` §1 (voir la fiche `carte-fonctionnelle-us-route-api-dto.md`).
- Toutes les décisions différées : `docs/BACKLOG.md`.
- Décisions de sécurité écartées ou datées : `SECURITY.md` §2 et §6.

À retenir

1. Le MVP obligatoire (US01 à US06) est livré et testé ; l'optionnel est partiel : US09 livré, US10 partiel, US07 et US08 non traités.
2. Le backlog n'est pas une liste de regrets : chaque entrée donne l'état, la raison, le filet actuel et le déclencheur.
3. Découper permet de différer à moindre coût : US10 a sa logique testée, il ne manque que le `@Scheduled`.
4. Rotation de clé JWT écartée à dessein : avec des tokens courts et sans refresh token, elle dégrade l'expérience ; un secret stable géré en environnement suffit pour le MVP.
5. À l'oral : nommer le manque, donner la raison, montrer le filet, dire ce qui déclencherait le travail.